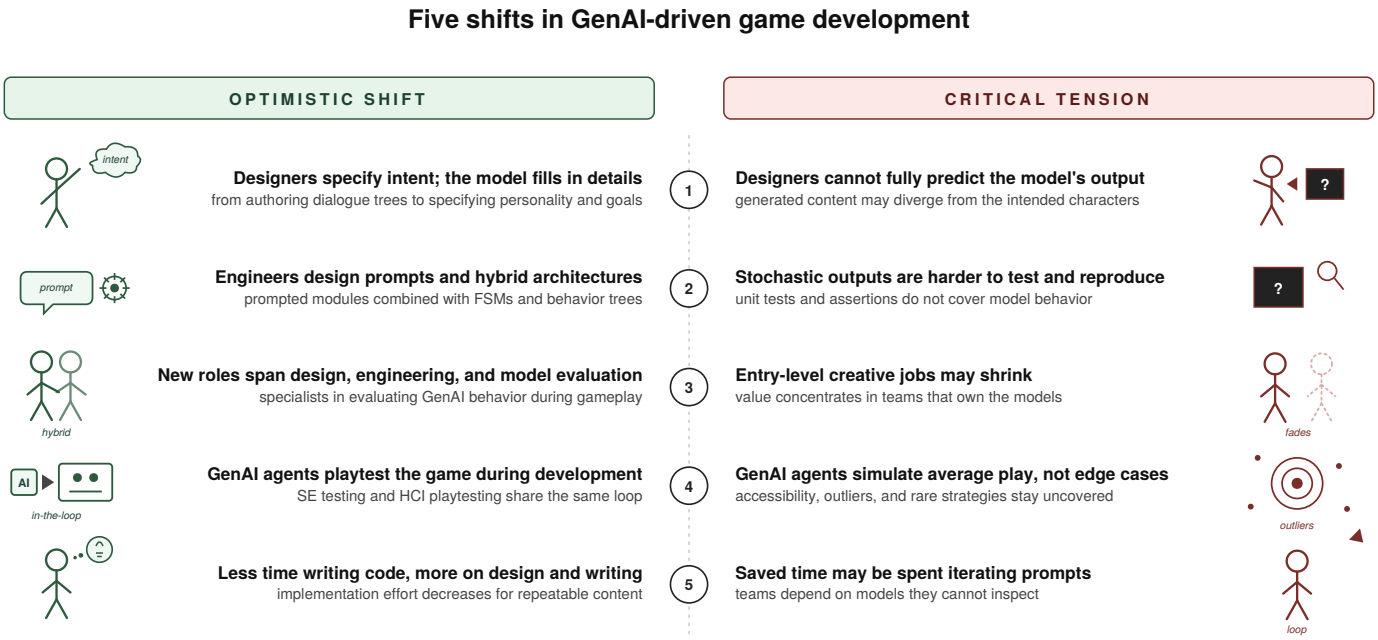
“We Must Be Better” A Critical Vision for the Convergence of Software Engineering and Human-Computer Interaction in Generative AI-Driven Video Game Development

Stefano Lambiase
Aalborg University Copenhagen
Copenhagen, Denmark
stla@cs.aau.dk



***Abstract*—Generative AI has the potential to profoundly change video game development, but its impact extends beyond the product to the process itself, opening the possibility of transforming the sector and making it even more creative than it already is. Three research traditions—i.e., game software engineering, generative AI in games, and simulated users in software testing—converge on the same object of study but operate in disciplinary silos with limited cross-fertilization. I argue that this is a missed opportunity, given the risk of failing to develop a shared vocabulary and of producing partial, scattered, and non-transferable knowledge. I envision, optimistically, that if leveraged carefully, generative AI can be the moment that brings game research to convergence, mirroring in the research community the transdisciplinary integration that already characterizes industrial practice. To discuss this, the paper adopts the critical review methodology, extended to generate a constructive vision: a shift that redefines designer and engineer roles around prompt and constraint design, opens new hybrid professional roles, and converges software engineering testing with Human-Computer Interaction playtesting. I argue this has the potential to reshape the field and to open extensive research lines, both on the constructive side and on the structural risks the shift raises.**

***Index Terms*—video game, software engineering, generative AI, large language models, testing, human-computer interaction**

Accepted version. This paper has been accepted for publication in the Proceedings of the IEEE Conference on Games (CoG) 2026; the final version of record will appear in IEEE *Xplore* (DOI to be added upon publication).

979-8-3315-9476-3/26/$31.00 ©2026 IEEE

I. Introduction

In *God of War*, Kratos turns to his son Atreus and says *“We must be better”* [1]. The line captures a stance that combines honest acknowledgment of past failure with the obligation to act differently going forward, without giving in to cynicism

or pretending that the past did not happen. I use it here because it reflects the position that the community can take toward research on generative artificial intelligence (GenAI) in video game software engineering (VGSE). Software Engineering (SE) research on games has remained a consolidating sub-area [2], [3], while a parallel human-computer interaction (HCI) tradition has been engaging with games user research and the industry's playtesting practices for over two decades [4]–[6]. GenAI introduces real risks alongside its opportunities, and the existing literature is already producing fragmented signals of a convergence it has not yet articulated. I argue that this is the moment for SE to engage with the games domain as a core contributor.

SE research on video games is neither absent nor dormant. Foundational work [7] and recent literature reviews [8] document a body of knowledge on game development practices, in which testing remains the bottleneck most resistant to automation. What this body of literature has *not* achieved, when compared to the parallel HCI tradition, is a comparable integration with industrial practice and a comparable visibility in the venues where games are designed and released [2], [4], [5], [9]. This asymmetry is structural rather than moral, and it shapes which research questions can be posed and which evidence is accepted as relevant.

The integration of GenAI changes the substrate of game development. Surveys document a rapidly expanding application space [10]–[13], and world models have recently extended this space further [14]. The resulting artifacts are stochastic and emergent, and SE practices of verification, regression detection, and deterministic debugging do not transfer cleanly to them. At the same time, the two communities, HCI and SE, are already converging in practice. Xiao and Yang [15] (HCI) and Chen, Habchi, and Wei [16] (SE) already build on the same agent substrate for converging ends, without citing each other. This convergence builds on earlier foundations: model-driven studies [17], [18] established the feasibility of simulated players before the GenAI wave, and broader empirical work [19] shows that LLMs can replicate aggregate human decision-making.

I argue that GenAI is the moment when SE rejoins games research as a core contributor to a paradigm shift in how game experience is produced. Read against Ralph and Monu's unified theory of digital games [20], GenAI moves production along the embedded-to-emergent axis. Gameplay and worlds are increasingly mediated through prompts and constraint architectures, which are used to elicit creative experiences at runtime; testing practice must change as a direct consequence. Validating these systems requires agents that emulate human behavior, and this calls for a combined SE–HCI focus.[1]

The paper follows a critical review methodology [21] and makes three main contributions. **First**, I extend the typology of LLM roles in games proposed by Gallotta et al. [10], which covers seven roles spanning gameplay and content creation, with an eighth role, *agent-as-tester*, explicitly situated at the SE–HCI boundary. **Second**, I articulate five interconnected shifts in game development practice: the emergence of new artifact classes, the redefinition of designer roles, the redefinition of engineer roles, the SE–HCI testing convergence as the structural linchpin, and the redistribution of professional and creative labor. I also discuss the tensions associated with each of these shifts. **Third**, I propose meta-scientific recommendations and a research agenda with concrete questions grounded in the current literature. Moreover, I adopt a deliberately optimistic stance, declared upfront, and I balance it with an explicit pessimistic counter-position for each major claim.

II. A Critical Review of Three Converging Research Traditions

This section reports on the method to develop the vision.

A. Methodological Commitment

I adopt the critical review methodology defined by Ralph and Baltes [21], which positions the contribution within the *meta-scientific discourse* of the research community—i.e., the conversation a community has internally about how it conducts and communicates research. Following their guidelines, a critical review proceeds by strategic sampling rather than exhaustive coverage, refrains from quality-assessing individual studies, and offers pointed critiques while maintaining community cohesion, citing specific works only where the citation supports a constructive direction, and avoiding naming-and-shaming as a rhetorical mode. Furthermore, instead of stopping at meta-scientific critique, I use it to generate a constructive vision (Sections III–IV) of the pattern I identify.[2]

Operationally, I drew a strategic sample from venues that publish at the intersection of VGSE, generative AI in games, and simulated users in software testing—e.g., IEEE Conference on Games, Foundations of Digital Games, IEEE Transactions on Games, *Information and Software Technology*, *Empirical Software Engineering*, *IEEE Transactions on Software Engineering*, CHI, and ICSE—over the time window 2014–2026. I included peer-reviewed papers and systematic or scoping reviews directly addressing one of the three traditions, and I admitted preprints from established research groups when they constituted the most recent available evidence on a fast-moving topic.

My sampling strategy led to three limitations. First, the sample is strategic and reflects my optimistic positioning; readers operating from a different stance would weight the corpus differently. Second, the field evolves rapidly enough that any closed corpus is provisional. Third, I illustrate problematic patterns without identifying individual papers as representative of those patterns. Nevertheless, these choices follow the strategic-sampling logic of Ralph and Baltes [21].

[1] I use the en-dashed compound *SE–HCI* to denote the methodological convergence between SE testing on the artifact side and HCI playtesting on the user side; the term is developed systematically in Section III-D.

[2] I further situate the paper with respect to the ACM SIGSOFT Empirical Standards [22]. I adhere to the *General Standard* and I do not claim conformance to the *Systematic Review Standard*, because that standard requires a step-by-step replicable search process as an essential attribute, whereas the critical review type explicitly endorses strategic sampling.

B. Three Silos in the Current Discourse

Game software engineering. A first research tradition has consolidated a body of knowledge on the practices, challenges, and differences that distinguish game development from general SE. Foundational comparative work [7] and follow-up consolidation efforts [2], [23] document recurring patterns: (i) tooling fragmentation, scope volatility, and crunch as failure modes that resist generic SE solutions [9], [24]; (ii) agile adoption as partial and complex [25]; and (iii) testing as the bottleneck most resistant to automation [26]–[28]. Regarding this last point, the most recent review in this field [8] confirms the persistent dominance of manual playtesting in industry and the relative immaturity of automated approaches; GenAI-driven testing is acknowledged as emergent but not yet systematized. Within the same tradition, the Cetina–Arcega line of work (e.g., [17], [18]) has demonstrated technical feasibility of simulated-player testing in software models of games for bug-localization purposes using rule-based agents, paving the way for the latest GenAI agents. ⚠ **The critical observation I draw is not that this tradition lacks rigor—it does not—but that its engagement with GenAI specifically remains SE-focused.** Simulated players are framed as instruments for code-level defect finding, not as a methodological bridge to the player-experience evaluation practices—central in game testing—that HCI has long owned.

Generative AI in games. A second tradition, anchored in AI and game-studies venues, has produced a wave of surveys and scoping reviews mapping the application space of large language models and other generative models in games [10]–[13]. The most influential of these [10] proposes a typology of seven LLM roles in games—i.e., player, non-player character, assistant, game master, designer, analyst, and commentator—and uses it to organize a roadmap of open challenges. ⚠ **The critical observation I draw is structural rather than evaluative. Every role in the typology is framed within gameplay or content creation; none addresses the role of *agent-as-validator* during development.** Scoping reviews in this tradition describe applications but rarely ask how to evaluate them empirically; only HCI-adjacent work [29], [30] engages this question. The literature is therefore rich in artifacts but thin on evaluation methodology—a gap the HCI tradition could fill, but rarely does, because cross-citation between the two communities is sparse.

Simulated users in software testing. A third tradition, mature in both classical SE and HCI research, has accumulated decades of work on simulated users, first as test agents for interactive systems, more recently as LLM-based proxies for usability and A/B evaluation [31]–[34]. Crucially, this community has internalized the limits of simulation as a first-order epistemic concern; demographic bias, behavioral fidelity, and external validity are recurring topics in its self-reflection [35], [36], and the prevailing recommendation converges on hybrid human-agent protocols as best practice rather than a wholesale replacement of human evaluators. ⚠ **The critical observation I draw is that this body of knowledge has not been systematically transferred to game contexts, despite being directly applicable.** For instance, this tradition's hard-won results on demographic bias and external validity in simulated users bear directly on whether GenAI agents can stand in for players, yet rarely reach the games venues where that substitution is now being proposed. The two works that come closest to bridging the gap [15], [16] run LLM agents on the same substrate but pursue divergent objectives in different venues, without citing each other.

The Meta-scientific Gap

The pattern that emerges from reading the three traditions side by side is not that any of them is wrong, methodologically weak, or insufficiently productive; it is that they pursue overlapping objects of study with non-overlapping methodologies, vocabularies, and venues, and that this fragmentation produces fragile evidence, missed opportunities for cross-fertilization, and a discourse that does not yet articulate what it is collectively converging toward. The recent parallel emergence of agent-based testing on the HCI side [15] and on the SE side [16], with no explicit awareness of the convergence and no shared vocabulary for it, is symptomatic of the gap rather than incidental to it. To make the divergence concrete, the same artifact—an agent that plays the game—is a *player proxy* evaluated by correlation with human difficulty perception on the HCI side and a *coverage maximizer* evaluated by code and interaction coverage on the SE side; even *validity* splits, denoting the ecological validity of a playtest in one tradition and the construct or external validity of an empirical study in the other. I name this the *meta-scientific gap*, i.e., a gap in the meta-scientific scaffolding—e.g., citation patterns, evaluation criteria, and methodological vocabulary—that would allow artifacts to be read and reviewed coherently across the three traditions.

III. A Convergent Vision

This section reports the envisioned shift in video game development as a system of causally connected transformations.

My vision (graphically represented in Figure 1) can be summarized as follows: the arrival of GenAI as a substrate for qualitatively new game artifacts (Section III-A) redefines what designers (Section III-B) and engineers (Section III-C) do; the same substrate enables an in-the-loop convergence between SE-style testing and HCI-style playtesting through GenAI agents (Section III-D); and the resulting practice rearticulates roles and labor, opening new hybrid professional positions and shifting work toward more interpretive curation (Section III-E).

I present the vision optimistically by design; I pair each major claim with its strongest pessimistic counter-position in context, marked with ϟ and set in a *tension* box next to the claim it qualifies. Moreover, my vision is bounded in scope. Specifically, I focus on narrative- and simulation-driven

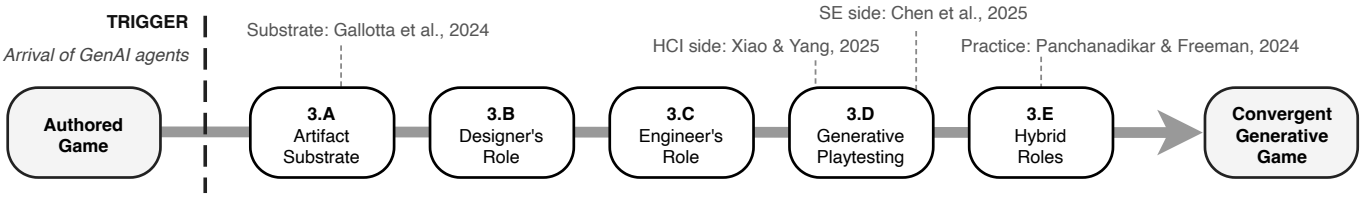


Fig. 1. The convergent vision presented in Section III.

games where the embedded-to-emergent shift has the strongest leverage; competitive multiplayer, where determinism and fairness constraints dominate, requires distinct considerations. Furthermore, I assume computational resources adequate for integrating GenAI into development pipelines; indie contexts may find the vision partially applicable, particularly the role and labor shifts. Nevertheless, whether the vision extends to indie and lower-budget contexts depends on the cost trajectory discussed in Section III-D—a pricing equilibrium with model vendors that is far from settled, and one that rising inference costs could push out of reach for smaller studios. Moreover, the argument targets generative technology in general and does not hinge on any specific model class.

A. Generative AI as Substrate for New Game Artifacts

GenAI introduces into game development a new set of artifacts—i.e., NPCs with persistent personalities, adaptive worlds, and emergent narrative—and what makes this class *qualitatively new* are three structural properties that distinguish them from the artifacts of classical VGSE.

First, artifact behavior—e.g., NPC dialogues and actions—is partially specified at design time and partially elicited at runtime. A behavior tree or finite state machine determines the artifact's response space at compile time, while a GenAI-driven NPC carries an open-ended response space whose realization depends on input, prompt, model state, and conversation history. Therefore, the artifact is not fully specified by its source but co-produced by source and runtime conditions in a way classical SE artifacts are not [10], [12].

Second, prompts and constraint specifications become first-class engineering artifacts. They are versioned, reviewed, and maintained; they encode semantic intent in structural form; and they change the behavior in ways sensitive to surrounding context. Treating them as text strings appended to chat completions misses what they actually do, which is to encode the engineering intent that classical structures used to encode in source code [10].

Third, the boundary between authored and emergent content—in Ralph and Monu's vocabulary [20], between embedded and emergent narrative—becomes fluid rather than fixed. The same character can carry authored backstory and emergent dialogue, and the same world can carry authored geography and emergent events [14], [29]. This fluidity is the feature, and it is also what makes these artifacts harder to specify, evaluate, and reason about with classical SE methods, which is the entry point for the role and method shifts that the rest of this section describes.

Tension—Accountability and Opacity

Because these artifacts are co-produced at runtime, their decision-making is partially opaque to the developers who build them. When a GenAI-driven NPC produces output that is offensive or inaccurate, responsibility is hard to assign and this opacity complicates the engineering accountability that classical SE has.

B. The Shift in the Designer's Role

As GenAI artifacts shift behavior toward runtime elicitation, the designer's role shifts with them. Where the classical game designer authored embedded narrative directly—e.g., dialogue trees branching per NPC, scripted events ordered along storyline beats, and fixed quest structures—the designer of a generative game articulates intent at a higher level, specifying global vision, character personality, narrative constraints, world-building principles, and the tonal envelopes that GenAI systems should respect at runtime [29], [37]. **In Ralph and Monu's vocabulary [20], the designer shifts from authoring embedded narrative directly to specifying the conditions under which emergent narrative plays out at runtime.** The narrative arc, the character's voice, and the world's logic are designed at design time, while specific dialogue lines, event sequences, and local variations are elicited at runtime within those conditions.

The above-mentioned shift is already partially observed in practice. Indie developers report integrating GenAI tools into design workflows in ways that absorb formerly specification-bound work into a more curatorial mode, iterating on prompts and personality profiles where they previously iterated on dialogue text [30]. Furthermore, similar shifts toward higher-level intent specification with GenAI scaffolding are documented in adjacent design tasks like character illustration [38].

C. The Shift in the Engineer's Role

The engineer's role undergoes a complementary shift. Implementing a behavior tree, finite state machine, or scripted decision tree gives way to a different family of practices, i.e., prompt engineering, the design of constraint systems that bound generative output, and the architecture of hybrid probabilistic-deterministic agentic systems where classical and

generative components interact.[3] 💡 **The engineer's task becomes building the runtime architectures through which designer-specified intent transforms into emergent behavior, designing prompts and constraints as systems rather than as ad-hoc strings.**

Recent vision work in mainstream SE anticipates the same shift outside the games domain. The GENIUS project [39] articulates how the generative turn restructures the SE life cycle around model-driven and prompt-mediated artifacts, and identifies oracle reliability and verification as the SE problems the shift opens up. Moreover, within games specifically, surveys document the breadth of LLM applications and the engineering challenges they introduce [10], while early SE-on-games-with-AI work demonstrates that simulated agents can already be integrated into model-driven engineering pipelines for purposes such as bug localization [17], [18].

I do not predict the disappearance of deterministic structures, since behavior trees, finite state machines, and scripted logic will likely survive alongside generative components for reasons of cost, controllability, and determinism—e.g., a competitive boss fight or a high-stakes scripted cutscene benefits from deterministic implementation in ways no probabilistic substrate can match. The shift I describe is therefore additive and hybrid rather than substitutive, and the engineer's repertoire expands rather than swaps.

⚡ Tension—Reproducibility and Verifiability

Because generative artifacts are non-deterministic, testing, debugging, and regression detection become harder to apply than with the deterministic structures they replace [39]. Model-driven simulated-player testing [17], [18] relies on deterministic re-execution to localize bugs, and assessment frameworks for automated game testing treat reproducibility as a central criterion [27]. The stochasticity reframing introduced below (Section III-D) only partially addresses this, since it does not recover oracle-based regression detection.

D. The SE–HCI Convergence via In-the-Loop Testing with Generative AI Agents

Two recent works mark an emerging convergence between two fields. From the HCI side, Xiao and Yang [15] demonstrate that LLM agents prompted with chain-of-thought reasoning correlate strongly with the human distribution of difficulty perception across puzzle and deck-building games, framing LLM agents as proxies for empirical playtesting; from the SE side, Chen, Habchi, and Wei [16] use personality-conditioned LLM agents to maximize code and interaction-level coverage during automated game testing, framing the same technical substrate as a coverage maximizer for QA. The two works do not cite each other, and neither articulates the convergence. 💡 **Nonetheless, they jointly point toward a practice in which GenAI agents serve as a shared substrate for HCI-flavored player-experience evaluation and SE-flavored automated coverage testing in synchronous development cycles, supporting both game design and QA.**

The convergence is accompanied by a theoretical move that classical SE has been slow to make. Stochasticity in generative agents has been treated as a limitation by mainstream SE—a barrier to oracle-based testing, a source of regression noise, an obstacle to reproducibility [39]—and I propose to reframe it in the HCI register by measuring *distributions* of outcomes, or *gameplay trajectories*,[4] instead of deterministic oracles. Xiao and Yang [15] demonstrate the move empirically, showing that their agents correlate with the human distribution of difficulty perception even when their absolute performance diverges from the human optimum. The flip from oracle to distribution is what licenses the convergence. Without it, SE testing and HCI playtesting cannot share a substrate, because the SE side requires deterministic comparison and the HCI side requires distributional comparison; with the flip in place, however, both sides converge on the same evaluation logic.

Within Gallotta et al.'s typology of LLM roles in games [10]—i.e., player, non-player character, assistant, game master, designer, analyst, and commentator—all seven roles are framed within gameplay or content creation, but none addresses the role of agent-as-validator during development. I propose an eighth role, *agent-as-tester*, situated explicitly at the SE–HCI boundary. The agent-as-tester is not a player, although it plays the game; it is not a designer, although it produces findings that inform design; and it is not an analyst, although it produces structured output. Rather, it is the role the development loop has been missing, and the role that generative playtesting establishes by design.

The technical components of generative playtesting already exist. Casamayor et al. [17] and Roca et al. [18] demonstrate that simulated-player testing in software models of games is feasible even pre-GenAI, with rule-based parametric agents successfully co-evolving with scenarios for bug localization, while MIMIC [16] operationalizes personality-driven diversity using a 7-trait model derived from the PathOS framework, showing that the GenAI substrate can carry personality-conditioned behavior coherently across coverage scenarios. The two strands together demonstrate that the technical substrate is in place; what was missing is the integrative SE–HCI framing this paper proposes, and the methodological discipline of treating distributional alignment with human players—rather than matching against deterministic oracles—as the validation criterion for the practice.

[3] I use *prompt* and *prompt engineering* as the current vocabulary for designating constraint specifications passed to generative components, but the vision does not depend on these technologies; whatever artifacts succeed prompts as the dominant interface to generative components, the substantive shift remains the same—the move from deterministic and tightly constrained implementation to a more open engineering practice.

[4] I introduce the term *gameplay trajectory* to denote the complete chain of actions, random events, and system responses that occur during a single simulated playthrough; what an agent population generates over a run of evaluations is then a distribution of trajectories rather than a single deterministic output, and the question becomes whether this distribution aligns with the distribution that a comparable population of human players would produce.

Tension—False Confidence in GenAI-Driven Testing

The deeper risk is the gap between what GenAI agents simulate and what real players do. Agent gameplay clusters around average behavior and misses the tails of the distribution, where outliers and accessibility needs sit [35], [36]; the danger is not that agents are wrong on average, but that they are right on average and wrong where it matters most. Hybrid human-agent protocols [32], [33] are the best practice in adjacent fields to solve this.

Tension—Cost and Sustainability

Integrating GenAI into the development pipeline also carries the cost of running it at scale. Training and serving large generative models have non-trivial computational and environmental costs [40], [41], and these are unevenly distributed. Studios able to run large models gain the benefits of the convergence more easily than those that cannot, deepening the value-concentration concern raised below (Section III-E). Energy-efficient *small language models* are one mitigation whose trade-offs the agenda of Section IV can usefully investigate.

E. Hybrid Roles and the Redistribution of Labor

The shifts in designer and engineer roles do not merely re-label existing functions; they make space for new hybrid roles whose work was previously distributed across designer and engineer divisions, or did not have a clear professional home at all. When prompts and constraint specifications become first-class artifacts (Section III-A), the practitioners who write and maintain them become identifiable as a profession, and I frame the emergence as rearticulation of work rather than as net replacement of one role by another.

In the following, I argue that a small taxonomy of emerging roles is already discernible. *Narrative AI specialists* own prompt and persona design for NPCs and game masters, taking on the work of operationalizing character voice and narrative tone that previously lived inside dialogue authoring. *Agent-evaluation specialists* design and operate the generative playtesting and coverage protocols that Section III-D describes, requiring fluency in both HCI evaluation methodology and SE testing rigor. *Constraint architects* specify the safety, consistency, and tonal envelopes within which GenAI components produce content, and the rule systems that bound model behavior at runtime. A fourth, more cautious category is the *designer-engineer hybrid* whose work spans intent specification and architectural design without belonging entirely to either tradition.

These roles also reshape the character of labor across the development pipeline. Generally, less time is spent on the granular implementation of deterministic structures and more time is spent on intent specification, prompt and constraint design, evaluation of stochastic outputs, and the curation tasks that the new artifact classes require. **Where classical implementation was largely mechanical—translating specifications into code through patterns the practitioner had internalized—the new work is interpretive, with judgment exercised over selection, calibration, and fit to vision.**

The rearticulation is partially observable in current practice. Broader analyses of GenAI in game design frame the role-redefinition as a structural feature of the field rather than a transitional artifact [29], and qualitative studies of indie developers document partial absorption of the new practices in real workflows—designers spending more time on prompt iteration than on direct authoring, engineers spending more time on integration and constraint design than on ground-up implementation [30]. What is interpretive in the new role is the act of curation, where the developer becomes a curator and integrator of stochastic outputs whose value depends on judgment rather than execution.

Tension—Deskilling and Value Redistribution

Deskilling has been documented in creative trades, especially at entry-level positions in concept art and narrative design [29]. Role rearticulation and role displacement can coexist, and value tends to concentrate in those who control the models. Whether the redistributed labor described above is creative or menial therefore depends on tooling and studio organization—factors the community can shape, but that the vision alone cannot guarantee.

IV. Conclusion and Future Research Agenda

This section presents meta-scientific recommendations to the community and a set of concrete research questions.

A. Meta-scientific Recommendations to the Community

Cross-tradition citation as a quality criterion. Reviewers and program committees at venues that sit at the intersection of VGSE, generative AI in games, and simulated users in testing—e.g., CoG, FDG, CHI PLAY, and ASE—should treat the lack of cross-citation between these literatures as a methodological weakness. Authors proposing GenAI-based testing tools for games should show engagement with HCI evaluation methodology, and authors proposing GenAI-based playtesting frameworks should show engagement with SE testing rigor. The goal is not an exhaustive bibliography, but visible engagement with adjacent traditions in the manuscript.

Distributional rather than oracle-based validation. Submissions evaluating GenAI agents as player proxies should report how well their agents match the distribution of human players, including variance, bias profiles, and demographic coverage; single-point performance metrics against deterministic oracles are not sufficient. This kind of distributional evaluation is common in HCI but still rare in SE-side work [35], and its adoption would mark a real step toward convergence.

Dual-purpose acknowledgment. Papers proposing GenAI agent frameworks for games should state clearly whether

the framework targets SE-internal QA (coverage, defect detection), HCI-flavored playtesting (difficulty, balance, experience), or both. Dual-purpose use is technically possible, but the two purposes are methodologically distinct, and conflating them creates confusion about what a framework can and cannot validate [15], [16].

Reporting transparency for stochastic systems. Papers using GenAI agents in games should report the exact prompt templates, personality trait operationalizations, model versions, and temperature settings used in the study. These reporting practices are in line with the broader empirical standards in software engineering research [22], [42] and with calls for reproducibility in machine learning [43]. Adopting them is one of the few concrete responses available to the reproducibility tension discussed in Section III-C.

B. Research Agenda

The research questions below follow from the tensions raised across the vision (Section III) and the recommendations in Section IV-A. They describe concrete empirical work that the community can undertake in the next few years.

Behavioral fidelity. Correlation with human difficulty perception has already been established [15]. However, correlation is not the same as distribution match, and demographic biases [35] can be hidden inside aggregate correlation values. Therefore, it is important to investigate whether GenAI player simulators reproduce the variance of human behavior in subjective qualities such as difficulty, fun, and immersion, within domain-specific tolerance bounds. Controlled studies with human ground truth and a varied agent population, comparing distributions rather than averages, are needed.

> ❓ **RQ_1**—Under what conditions do generative AI player simulators reproduce the variance of human player behavior on subjective game qualities within domain-specific tolerance bounds?

Pipeline integration. The anchor works [15], [16] do not address how their methods would fit into iterative development workflows. This is an engineering question, but also a business one gated by time and resource budgets and by the return a studio expects. In-the-loop testing with GenAI agents needs to be inserted into game development pipelines in such a way that playtesting and coverage signals inform design decisions in synchronized cycles, rather than as batch outputs at the end of a sprint. Case studies on prototype tooling, action research with studio partners, and longitudinal observation of how teams adopt or reject such integration are needed.

> ❓ **RQ_2**—How can in-the-loop testing with generative AI agents be integrated into iterative game development workflows so that playtesting and coverage signals inform design decisions in synchronized cycles, and under what time and resource constraints is this viable for a studio?

Personality conditioning. Personality-conditioned LLM agents have been shown to improve code coverage [16]. It is still unclear whether this gain extends to behavioral fidelity, that is, to correlation with human distributions of subjective qualities. The open question is whether personality-conditioned agents—built on player models such as PathOS or on trait taxonomies adapted from other simulated-user contexts—provide better fidelity than non-conditioned agents, and for which tasks and game genres these gains hold. Underlying RQ_3—and RQ_4 below—is a construct-validity concern. *Personality* and *behavior* are subjective notions, and comparing results across studies presupposes shared, acceptable operationalizations of them. Establishing common definitions—and tooling for deriving consistent personalities and behaviors from GenAI across models and games—is therefore a prerequisite; without such a baseline, quantitative evaluation of these qualitative traits is hard to interpret. Replicating and extending MIMIC with human ground truth on subjective measures would help answer this question.

> ❓ **RQ_3**—Do personality-conditioned large-language-model agents provide better behavioral fidelity to human player populations than non-conditioned agents, beyond their established benefit on test coverage?

Limits, bias, and cost trade-offs. The false-confidence tension of Section III-D calls for an empirical mapping of what GenAI player simulators *cannot* surface. Documented limits include outliers [35] and demographic underrepresentation [36], but the specific game-design and accessibility scenarios where these gaps matter most have not yet been inventoried. The question becomes even more relevant when the cost tension of Section III-D is taken into account. Smaller and more energy-efficient generative agents, such as small language models, distilled architectures, and on-device inference, trade scale for efficiency and may make underrepresentation worse. I therefore propose RQ_4 as the empirical investigation of both axes together. The goal is to identify what is missed under frontier-scale agents, what is additionally missed under their smaller and more efficient counterparts, and how hybrid human-agent protocols can compensate in both cases.

> ❓ **RQ_4**—What classes of player behavior, edge case, or accessibility scenario are systematically underrepresented in generative AI player simulators—across both frontier-scale and small, energy-efficient model regimes—and how can hybrid human-agent protocols compensate?

Practices and roles. The hybrid roles taxonomy and the labor redistribution discussed in Section III-E are hypotheses about how studios will reorganize. Whether and how this reorganization actually happens, what its consequences are, and what new artifacts—e.g., prompts, persona specifications, agent populations, evaluation suites—emerge as engineering deliverables in their own right are all empirical questions.

Interview studies with practitioners across studio sizes, post-mortem analyses of completed productions, and ethnographic observation of teams in transition would help answer them.

> ❷ **RQ$_5$**—How are roles, workflows, and responsibilities redistributed in game studios adopting generative AI agents in their development pipelines, and what new artifacts emerge as engineering deliverables?

The five research questions above describe how the stance of this paper can be turned into empirical work. My hope is that this vision will endure and be supported by the research community, leading to a community that—as the opening quote of the paper reminds us—can be better.

Acknowledgment

The author thanks his cousin Alfonso Cavallo and his friend Giovanni Taiani, who informally reviewed the paper. Moreover, the author thanks his colleague Stina Olsson, who helped with the cover figure for the paper.

References

[1] Santa Monica Studio, "God of War," Sony Interactive Entertainment, PlayStation 4, 2018.

[2] J. Chueca *et al.*, "The consolidation of game software engineering: A systematic literature review of software engineering for industry-scale computer games," *Information and Software Technology*, vol. 165, p. 107330, 2023.

[3] F. Almansoury, S. Kpodjedo, and G. El Boussaidi, "Game development topics: A tag-based investigation on game development stack exchange," *Applied Sciences*, vol. 12, no. 21, p. 10750, 2022.

[4] P. Mirza-Babaei *et al.*, "Games user research: Practice, methods, and applications," in *CHI '13 Extended Abstracts on Human Factors in Computing Systems*. ACM, 2013, pp. 3219–3222.

[5] R. Bernhaupt, Ed., *Evaluating User Experience in Games: Concepts and Methods*, ser. Human-Computer Interaction Series. Springer, 2010.

[6] A. Denisova *et al.*, "Towards democratisation of games user research: Exploring playtesting challenges of indie video game developers," *Proceedings of the ACM on Human-Computer Interaction*, vol. 8, no. CHI PLAY, p. Article 343, 2024.

[7] E. Murphy-Hill, T. Zimmermann, and N. Nagappan, "Cowboys, ankle sprains, and keepers of quality: How is video game development different from software development?" in *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*. ACM, 2014, pp. 1–11.

[8] A. Roque *et al.*, "A literature review of software testing practices and frameworks in the video gaming industry," *Software Testing, Verification and Reliability*, vol. 35, 2025.

[9] C. Politowski *et al.*, "Game industry problems: An extensive analysis on the gray literature," *Information and Software Technology*, vol. 134, p. 106538, 2021.

[10] R. Gallotta *et al.*, "Large language models and games: A survey and roadmap," *IEEE Transactions on Games*, 2024.

[11] D. Yang, E. Kleinman, and C. Harteveld, "Gpt for games: A scoping review (2020-2023)," in *2024 IEEE Conference on Games (CoG)*. IEEE, 2024, pp. 1–8.

[12] P. Sweetser, "Large language models and video games: A preliminary scoping review," in *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, 2024, pp. 1–8.

[13] A. Al-Ghamdi *et al.*, "Generative artificial intelligence in game design: A narrative review," *Journal of Ecohumanism*, vol. 4, no. 4, 2025.

[14] A. Kanervisto *et al.*, "World and human action models towards gameplay ideation," *Nature*, vol. 638, pp. 656–663, 2025.

[15] C. Xiao and Z. F. Yang, "LLMs may not be human-level players, but they can be testers: Measuring game difficulty with LLM agents," *Proceedings of the ACM on Human-Computer Interaction*, vol. 9, no. 6, pp. 1097–1123, 2025.

[16] Y. Chen, S. Habchi, and L. Wei, "MIMIC: Integrating diverse personality traits for better game testing using large language model," *arXiv preprint arXiv:2510.01635*, 2025.

[17] R. Casamayor *et al.*, "Boosting bug localization in software models of video games with simulations and component-specific genetic operations," *Software and Systems Modeling*, vol. 24, pp. 1157–1185, 2025.

[18] I. Roca *et al.*, "Co-evolving scenarios and simulated players to locate bugs that arise from the interaction of software models of video games," *Information and Software Technology*, vol. 169, p. 107412, 2024.

[19] E. Akata *et al.*, "Playing repeated games with large language models," *Nature Human Behaviour*, vol. 9, pp. 1380–1390, 2025.

[20] P. Ralph and K. Monu, "Toward a Unified Theory of Digital Games," *The Computer Games Journal*, vol. 4, no. 1, pp. 81–100, Jun. 2015. [Online]. Available: https://doi.org/10.1007/s40869-015-0007-7

[21] P. Ralph and S. Baltes, "Paving the way for mature secondary research: The seven types of literature review," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. Singapore, Singapore: ACM, 2022, pp. 1632–1636.

[22] P. Ralph *et al.*, "ACM SIGSOFT empirical standards for software engineering research," arXiv preprint arXiv:2010.03525, 2021.

[23] S. Aleem, L. F. Capretz, and F. Ahmed, "Game development software engineering process life cycle: A systematic review," *Journal of Software Engineering Research and Development*, vol. 4, pp. 1–30, 2016.

[24] C. Politowski *et al.*, "Dataset of video game development problems," in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20)*. ACM, 2020, pp. 553–557.

[25] T. McKenzie *et al.*, "Is agile not agile enough? a study on how agile is applied and misapplied in the video game development industry," in *Proceedings of the IEEE/ACM Joint 15th International Conference on Software and System Processes (ICSSP) and 16th ACM/IEEE International Conference on Global Software Engineering (ICGSE)*. IEEE, 2021, pp. 94–105.

[26] J. Kasurinen and K. Smolander, "What do game developers test in their products?" in *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '14)*. ACM, 2014, pp. 1–10.

[27] A. Albaghajati and M. Ahmed, "Video game automated testing approaches: An assessment framework," *IEEE Transactions on Games*, vol. 15, no. 1, pp. 81–94, 2023.

[28] R. E. S. Santos *et al.*, "Computer games are serious business and so is their quality: Particularities of software testing in game development from the perspective of practitioners," in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '18)*. ACM, 2018, pp. 1–10.

[29] S. A. Alharthi, "Generative ai in game design: Enhancing creativity or constraining innovation?" *Journal of Intelligence*, vol. 13, no. 6, p. 60, 2025.

[30] R. Panchanadikar and G. Freeman, "I'm a solo developer but ai is my new ill-informed co-worker: Envisioning and designing generative ai to support indie game development," *Proceedings of the ACM on Human-Computer Interaction*, vol. 8, no. CHI PLAY, pp. 1–26, 2024.

[31] S. Ariyurek, A. Betin-Can, and E. Surer, "Automated video game testing using synthetic and humanlike agents," *IEEE Transactions on Games*, vol. 13, no. 1, pp. 50–67, 2021.

[32] F. Pehar, "AI as synthetic users in human-centred design: A systematic review," in *Proceedings of the 48th MIPRO ICT and Electronics Convention*. IEEE, 2025, pp. 1500–1509.

[33] S. Liu *et al.*, "Free lunch for user experience: Crowdsourcing agents for scalable user studies," arXiv preprint arXiv:2505.22981, 2025.

[34] Y. Lu *et al.*, "AgentA/B: Automated and scalable web A/B testing with interactive LLM agents," *arXiv preprint arXiv:2504.09723*, 2025.

[35] P. Seshadri *et al.*, "Lost in simulation: LLM-Simulated users are unreliable proxies for human users in agentic evaluations," arXiv preprint arXiv:2601.17087, 2026.

[36] Y. Gao *et al.*, "Take caution in using LLMs as human surrogates: Scylla ex machina," *arXiv preprint arXiv:2410.19599*, 2024.

[37] A. Begemann and J. Hutson, "Empirical insights into ai-assisted game development: A case study on the integration of generative ai tools in creative pipelines," *Metaverse*, vol. 5, no. 2, 2024.

[38] L. Ling *et al.*, "Sketchar: Supporting character design and illustration prototyping using generative ai," *Proceedings of the ACM on Human-Computer Interaction*, vol. 8, no. CHI PLAY, pp. 1–28, 2024.

[39] R. Gröpler *et al.*, "The future of generative ai in software engineering: A vision from industry and academia in the european GENIUS project," in *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 2025, pp. 170–181.

[40] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*. ACL, 2019, pp. 3645–3650.

[41] D. Patterson *et al.*, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[42] S. Baltes *et al.*, "Guidelines for empirical studies in software engineering involving large language models," 2026. [Online]. Available: https://arxiv.org/abs/2508.15503

[43] J. Pineau *et al.*, "Improving reproducibility in machine learning research: A report from the neurips 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, pp. 1–20, 2021.